Document number: P0576R0
Date: 2017-02-06
Project: Programming Language C++
Reference: ISO/IEC IS 14882:2014
Reply to: William M. Miller
Edison Design Group, Inc.
wmm@edg.com

Core Language Working Group "tentatively ready" Issues for the February, 2016 (Kona) meeting

Section references in this document reflect the section numbering of document WG21 N4606.

2194. Impossible case in list initialization

Section: 13.3.1.7 [over.match.list] **Status:** tentatively ready **Submitter:** Robert Haberlach **Date:** 2015-11-04

According to 13.3.1.7 [over.match.list] paragraph 1 says,

> If the initializer list has no elements and `T` has a default constructor, the first phase is omitted.

However, this case cannot occur. If `T` is a non-aggregate class type with a default constructor and the initializer is an empty initializer list, the object will be value-constructed, per 8.6.4 [dcl.init.list] bullet 3.4. Overload resolution is only necessary if default-initialization (or a check of its semantic constraints) is implied, with the relevant section concerning candidates for overload resolution being 13.3.1.3 [over.match.ctor].

See also issue 1518.

Proposed resolution (January, 2017):

Change 13.3.1.7 [over.match.list] paragraph 1 as follows:

> When objects of non-aggregate class type `T` are list-initialized such that 8.6.4 [dcl.init.list] specifies that overload resolution is performed according to the rules in this section, overload resolution selects the constructor in two phases:
>
> - Initially, the candidate functions are the initializer-list constructors (8.6.4 [dcl.init.list]) of the class `T` and the argument list consists of the initializer list as a single argument.
> - If no viable initializer-list constructor is found, overload resolution is performed again, where the candidate functions are all the constructors of the class `T` and the argument list consists of the elements of the initializer list.
>
> ~~If the initializer list has no elements and `T` has a default constructor, the first phase is omitted.~~ In copy-list-initialization, if an explicit constructor is chosen...

2198. Linkage of enumerators

Section: 3.5 [basic.link] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2015-11-12

According to the rules in 3.5 [basic.link] paragraph 4, the enumerators of an enumeration type with linkage also have linkage. Having same-named enumerators in different translation units would seem to be innocuous. Is there a rationale for this rule?

Proposed resolution (January, 2017):

Delete 3.5 [basic.link] bullet 4.5:

> An unnamed namespace or a namespace declared directly or indirectly within an unnamed namespace has internal

linkage. All other namespaces have external linkage. A name having namespace scope that has not been given internal linkage above has the same linkage as the enclosing namespace if it is the name of

- ...
- ~~an enumerator belonging to an enumeration with linkage; or~~
- ...

2201. Cv-qualification of array types

Section: 3.9.3 [basic.type.qualifier] **Status:** tentatively ready **Submitter:** Robert Haberlach **Date:** 2015-11-15

Issue 1059 changed 3.9.3 [basic.type.qualifier] paragraph 6 to read,

> An array type whose elements are cv-qualified is also considered to have the same cv-qualifications as its elements.

However, that change overlooked the earlier statement in paragraph 2,

> A compound type (3.9.2 [basic.compound]) is not cv-qualified by the cv-qualifiers (if any) of the types from which it is compounded. Any cv-qualifiers applied to an array type affect the array element type, not the array type (8.3.4 [dcl.array]).

Proposed resolution (January, 2017):

Change 3.9.3 [basic.type.qualifier] paragraph 2 as follows:

> A compound type (3.9.2 [basic.compound]) is not cv-qualified by the cv-qualifiers (if any) of the types from which it is compounded. Any cv-qualifiers applied to an array type affect the array element type~~, not the array type~~ (8.3.4 [dcl.array]).

2206. Composite type of object and function pointers

Section: 5 [expr] **Status:** tentatively ready **Submitter:** Mike Miller **Date:** 2015-12-01

Consider an example like

```
void *p;
void (*pf)();
auto x = true ? p : pf;
```

The rules in 5 [expr] paragraph 13 say that the composite type between a `void*` and a function pointer type is `void*`. This is surprising, since a function pointer type cannot be implicitly converted to `void*`.

Proposed resolution (January, 2017):

Change 5 [expr] bullet 14.5 as follows:

> The *cv-combined type* of two types `T1` and `T2` is a type `T3` similar to `T1` whose cv-qualification signature (4.5 [conv.qual]) is:
>
> - ...
> - if `T1` or `T2` is "pointer to *cv1* `void`" and the other type is "pointer to *cv2* `T`", **where `T` is an object type or `void`,** "pointer to *cv12* `void`", where *cv12* is the union of *cv1* and *cv2*;
> - ...

2214. Missing requirement on representation of integer values

Section: 3.9.1 [basic.fundamental] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2015-12-15

According to 3.9.1 [basic.fundamental] paragraph 3,

> The range of non-negative values of a signed integer type is a subrange of the corresponding unsigned integer type, and the value representation of each corresponding signed/unsigned type shall be the same.

The corresponding wording from C11 is,

> The range of nonnegative values of a signed integer type is a subrange of the corresponding unsigned integer type, and the representation of the same value in each type is the same.

The C wording is arguably clearer, but it loses the implication of the C++ wording that the sign bit of a signed type is part of the value representation of the corresponding unsigned type.

Proposed resolution (January, 2017):

Change 3.9.1 [basic.fundamental] paragraph 3 as follows:

> ...The standard and extended unsigned integer types are collectively called unsigned integer types. The range of non-negative values of a signed integer type is a subrange of the corresponding unsigned integer type, **the representation of the same value in each of the two types is the same,** and the value representation of each corresponding signed/unsigned type shall be the same. The standard signed integer types...

2220. Hiding index variable in range-based `for`

Section: 6.5.4 [stmt.ranged] **Status:** tentatively ready **Submitter:** Daveed Vandevoorde **Date:** 2016-01-08

Given an example like

```
void f() {
  int arr[] = { 1, 2, 3 };
  for (int val : arr) {
    int val;   // Redeclares index variable
  }
}
```

one might expect that the redeclaration of the index variable would be an error, as it is in the corresponding classic `for` statement. However, the restriction that makes the latter an error is phrased in terms of the *condition* nonterminal in 6.4 [stmt.select] paragraph 3, and the range-based `for` does not refer to *condition*. Should there be an explicit prohibition of such a redeclaration?

Proposed resolution (January, 2017):

Add the following as a new paragraph before 6.5 [stmt.iter] paragraph 4:

> **If a name introduced in an *init-statement* or *for-range-declaration* is redeclared in the outermost block of the substatement, the program is ill-formed. [*Example:***
>
> ```
> void f() {
>   for (int i = 0; i < 10; ++i)
>     int i = 0;          // error: redeclaration
>   for (int i : { 1, 2, 3 })
>     int i = 1;          // error: redeclaration
> }
> ```
>
> ***—end example*]**
>
> [*Note:* The requirements on conditions in iteration statements are described in 6.4 [stmt.select]. *—end note*]

2224. Member subobjects and base-class casts

Section: 5.2.9 [expr.static.cast] **Status:** tentatively ready **Submitter:** Aaron Ballman **Date:** 2016-01-13

The current wording of 5.2.9 [expr.static.cast] paragraph 2 appears to permit the following example:

```
struct B {
  int i;
};

struct D : B {
  int j;
  B b;
};

int main() {
  D d;

  B &br = d.b;
  D &dr = static_cast<D&>(br);  // Okay?
}
```

Presumably such casts should only be supported if the operand object is a base class subobject, not a member subobject.

Proposed resolution (January, 2017):

Change 5.2.9 [expr.static.cast] paragraph 2 as follows:

> ...If the object of type "*cv1* `B`" is actually a **base** subobject of an object of type `D`, the result refers to the enclosing object of type `D`. Otherwise, the behavior is undefined. [*Example:*...

2259. Unclear context describing ambiguity

Section: 8.2 [dcl.ambig.res] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2016-04-12

According to 8.2 [dcl.ambig.res] paragraph 3,

> Another ambiguity arises in a *parameter-declaration-clause* of a function declaration, or in a *type-id* that is the operand of a `sizeof` or `typeid` operator, when a *type-name* is nested in parentheses.

There are two problems here: first, a *parameter-declaration-clause* appears in a *lambda-expression*, not just in a function declaration. Second, the ambiguity can arise in a *type-id* appearing in any context, not just in a `sizeof` or `typeid` expression.

Proposed resolution (January, 2017):

Change 8.2 [dcl.ambig.res] paragraph 3 as follows:

> Another ambiguity arises in a *parameter-declaration-clause* ~~of a function declaration, or in a *type-id* that is the operand of a `sizeof` or `typeid` operator~~, when a *type-name* is nested in parentheses. In this case, the choice is between the declaration of a parameter of type pointer to function and the declaration of a parameter with redundant parentheses around the *declarator-id*. The resolution is to consider the *type-name* as a *simple-type-specifier* rather than a *declarator-id*. [*Example:*...

2262. Attributes for *asm-definition*

Section: 7.4 [dcl.asm] **Status:** tentatively ready **Submitter:** Richard Smith **Date:** 2016-05-04

There does not seem to be a good reason not to permit attributes on an `asm` declaration. This would be handy for things like:

```
[[vendor::asm_syntax("intel")]] asm(...);
```

Notes from the December, 2016 teleconference:

The omission seems to have been an oversight that should be corrected.

Proposed resolution (January, 2017):

Change 7.4 [dcl.asm] paragraph 1 as follows:

An `asm` declaration has the form

asm-definition:
***attribute-specifier-seq*$_{opt}$** `asm` `(` *string-literal* `)` `;`

The `asm` declaration is conditionally-supported; its meaning is implementation-defined. **The optional *attribute-specifier-seq* in an *asm-definition* appertains to the `asm` declaration.** [*Note:* Typically it is used to pass information through the implementation to an assembler. —*end note*]